SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks: 30**

Annexure A **Dry Run**

Evaluation

Code of Conduct:

I T NITHIN KUMAR (192424351) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Student Name: T NITHIN KUMAR

Register Number:192424351

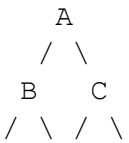
Date: 23/10/2026

Annexure A **Dry Run**
Evaluation

1. Question 1 – Breadth-First Search (BFS)

1.1 Question

Starting from node **A** of the graph given below, perform a Breadth-First Search (BFS). Complete the iteration table, write the BFS traversal order, and show the queue contents after each iteration.



D E F G

1.2 Aim

To traverse the given graph level by level from the root node A using BFS and to record the queue contents at every iteration.

1.3 Algorithm / Concept

BFS explores all nodes at the current depth before moving to the next depth. It uses a **queue** (First-In-First-Out). Children are added in **left-to-right** order.

Step 1: Insert the root node A into the queue.

Step 2: Remove the node at the front of the queue and mark it as visited.

Step 3: Add all its unvisited children (left to right) to the rear of the queue. Step

4: Repeat Steps 2 and 3 until the queue becomes empty.

1.4 Step-by-Step Dry Run

Initial queue: [A] (front on the left, rear on the right)

Iteration 1: Remove A; add B, C. Queue = [B, C]

Iteration 2: Remove B; add D, E. Queue = [C, D, E]

Iteration 3: Remove C; add F, G. Queue = [D, E, F, G]

Iteration 4: Remove D; D has no children. Queue = [E, F, G]

Iteration 5: Remove E; E has no children. Queue = [F, G]

Iteration 6: Remove F; F has no children. Queue = [G]

Iteration 7: Remove G; G has no children. Queue empty – stop. Queue = [ ] (empty)

1.5 Completed Table

The Queue column shows the contents of the queue after the iteration is completed.

Iteration	Queue (after iteration)	Visited Node
1	[B, C]	A
2	[C, D, E]	B
3	[D, E, F, G]	C
4	[E, F, G]	D
5	[F, G]	E
6	[G]	F
7	[ ] (empty)	G

1.6 Final Answer

(a) BFS traversal order:

A → B → C → D → E → F → G

(b) Queue contents after each iteration:

```
Initial : [A]
Iter 1  : [B, C]
Iter 2  : [C, D, E]
Iter 3  : [D, E, F, G]
Iter 4  : [E, F, G]
Iter 5  : [F, G]
Iter 6  : [G]
Iter 7  : [ ] (empty)
```

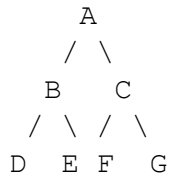
1.7 Explanation and Result

BFS visited the root A (level 0), then B and C (level 1), and finally D, E, F, G (level 2). Because the queue is FIFO, nodes inserted earlier are always visited earlier, so the traversal proceeds strictly level by level. All 7 nodes were visited exactly once in 7 iterations.

2. Question 2 – Depth-First Search (DFS)

2.1 Question

Using the same graph, perform Depth-First Search (DFS) starting from **A** and complete the table.



2.2 Aim

To traverse the graph by going as deep as possible along each branch before backtracking, and to record the stack contents at each step.

2.3 Algorithm / Concept

DFS follows one path to its deepest node, then backtracks. It uses a **stack** (Last-In-First-Out) or, equivalently, recursion, where the call stack plays the role of the explicit stack. The most recently discovered node is always explored first.

Stack convention: the **top of the stack is written on the left**. To visit children left to right, the children are pushed in **reverse order** (right child first, then left child), so that the left child is on top and is popped first.

Step 1: Push the root A onto the stack.

Step 2: Pop the top node and mark it as visited.

Step 3: Push its children in reverse order (right child first). Step

4: Repeat Steps 2 and 3 until the stack is empty.

2.4 Step-by-Step Dry Run

Initial stack: [A]

Step 1: Pop A; push C, then B. Stack = [B, C]

Step 2: Pop B; push E, then D. Stack = [D, E, C]

Step 3: Pop D; no children. Stack = [E, C]

Step 4: Pop E; no children. Stack = [C]

Step 5: Pop C; push G, then F. Stack = [F, G]

Step 6: Pop F; no children. Stack = [G]

Step 7: Pop G; no children. Stack = [ ] (empty)

2.5 Completed Table

Iteration	Stack before pop (top on left)	Visited Node	Stack after push (top on left)
1	[A]	A	[B, C]
2	[B, C]	B	[D, E, C]
3	[D, E, C]	D	[E, C]
4	[E, C]	E	[C]
5	[C]	C	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[ ] (empty)

2.6 Final Answer

DFS traversal order:

A → B → D → E → C → F → G

2.7 Explanation and Result

DFS first went from A to its left child B, then to B's left child D. After D (a leaf), it backtracked to B and visited E, then backtracked to A and explored the right subtree C, F, G. All 7 nodes were visited exactly once. Unlike BFS, which visits level by level (A, B, C, D, E, F, G), DFS follows each branch to its end first.

3. Question 3 – 8-Puzzle Problem Using BFS

3.1 Question

For the initial and goal states given below, trace Breadth-First Search (BFS) until the goal state is reached, fill in the table, and answer the four questions.

Initial State	Goal State
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

3.2 Aim

To find the shortest sequence of blank-tile moves that transforms the initial state into the goal state using BFS.

3.3 Algorithm / Concept

The **8-puzzle** is a 3 × 3 board with tiles 1–8 and one blank, represented by **0**. A **state** is one arrangement of the tiles. The initial state is **136 502 478** and the goal state is **123 456 780**.

Legal moves: the blank may move **Up, Down, Left or Right** by swapping with the adjacent tile, provided it stays inside the board. A blank in the centre has 4 moves, on an edge 3 moves, and in a corner 2 moves. Every move has cost 1 (unweighted).

BFS procedure: states are stored in a FIFO queue. The front state is removed and **expanded** (all legal successors are generated). Each successor is checked against a **visited set**; only new states are added to the rear of the queue. The search stops when the goal state is removed from the queue. **Operator order used throughout:** Up, Down, Left, Right.

Duplicate avoidance: the visited set stores every state ever placed in the queue. A successor already in this set (for example, the parent state obtained by undoing the last move) is discarded, so no state is queued or expanded twice.

Terminology for counts: a state is **expanded** when it is removed from the queue and its successors are produced; a successor is **generated** every time a legal move produces it, even if it is a duplicate; a state is **discovered** when it is generated for the first time and added to the queue/visited set.

Notation: states are named S0, S1, S2, … in the order they are discovered. S0 is the initial state. In BFS the states are expanded in this same order, so the queue at any time is a continuous block of numbers, e.g. [S5 – S14].

3.4 Step-by-Step Dry Run

The first five iterations are shown in full below.

Iteration 1: expand S0 = 136 502 478 (depth 0). Queue before: [S0].

```
Up: 106 532 478 → new, S1
Down: 136 572 408 → new, S2
Left: 136 052 478 → new, S3
Right: 136 520 478 → new, S4
```

Queue after: [S1 – S4] (4 states).

Iteration 2: expand S1 = 106 532 478 (depth 1). Queue before: [S1 – S4].

```
Down: 136 502 478 → duplicate (already visited)
Left: 016 532 478 → new, S5
Right: 160 532 478 → new, S6
```

Queue after: [S2 – S6] (5 states).

Iteration 3: expand S2 = 136 572 408 (depth 1). Queue before: [S2 – S6].

```
Up: 136 502 478 → duplicate (already visited)
Left: 136 572 048 → new, S7
```

```
Right: 136 572 480 → new, S8
```

Queue after: [S3 – S8] (6 states).

Iteration 4: expand S3 = 136 052 478 (depth 1). Queue before: [S3 – S8].

```
Up: 036 152 478 → new, S9
Down: 136 452 078 → new, S10
Right: 136 502 478 → duplicate (already visited)
```

Queue after: [S4 – S10] (7 states).

Iteration 5: expand S4 = 136 520 478 (depth 1). Queue before: [S4 – S10].

```
Up: 130 526 478 → new, S11
Down: 136 528 470 → new, S12
Left: 136 502 478 → duplicate (already visited)
```

Queue after: [S5 – S12] (8 states).

The remaining iterations follow the same procedure; their expansion log is given in Section 3.5.

3.5 Completed Table

Table A – Iterations 1 to 5 (original table). Queue entries are state numbers; the 3 × 3 states they stand for are listed in Table B.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 1 3 6 5 _ 2 4 7 8	[S0]	[S1, S2, S3, S4]
2	S1 1 _ 6 5 3 2 4 7 8	[S1, S2, S3, S4]	[S2, S3, S4, S5, S6]
3	S2 1 3 6 5 7 2 4 _ 8	[S2, S3, S4, S5, S6]	[S3, S4, S5, S6, S7, S8]
Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
4	S3 1 3 6 _ 5 2 4 7 8	[S3, S4, S5, S6, S7, S8]	[S4, S5, S6, S7, S8, S9, S10]
5	S4 1 3 6 5 2 _ 4 7 8	[S4, S5, S6, S7, S8, S9, S10]	[S5, S6, S7, S8, S9, S10, S11, S12]

(Blank tile shown as "_".)

Table B – States discovered in iterations 1 to 5.

State	State	State

S0 1 3 6 5 _ 2 4 7 8 initial	S1 1 _ 6 5 3 2 4 7 8 from S0 (Up)	S2 1 3 6 5 7 2 4 _ 8 from S0 (Down)
S3 1 3 6 _ 5 2 4 7 8 from S0 (Left)	S4 1 3 6 5 2 _ 4 7 8 from S0 (Right)	S5 _ 1 6 5 3 2 4 7 8 from S1 (Left)
S6 1 6 _ 5 3 2 4 7 8 from S1 (Right)	S7 1 3 6 5 7 2 _ 4 8 from S2 (Left)	S8 1 3 6 5 7 2 4 8 _ from S2 (Right)
S9 _ 3 6 1 5 2 4 7 8 from S3 (Up)	S10 1 3 6 4 5 2 _ 7 8 from S3 (Down)	S11 1 3 _ 5 2 6 4 7 8 from S4 (Up)
S12 1 3 6 5 2 8 4 7 _ from S4 (Down)		

Table C – Continuation: iterations 6 to 311 (complete expansion log). Current state S is the state expanded in that iteration. "New" is the number of newly discovered states added to the queue; "Gen." is the number of successors generated. The queue before expansion is the block S – S(last discovered); the queue after expansion starts from S(k+1).

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
6	S5	016 532 478	2	S5 – S12 (8)	2	1	S6 – S13 (8)
7	S6	160 532 478	2	S6 – S13 (8)	2	1	S7 – S14 (8)
8	S7	136 572 048	2	S7 – S14 (8)	2	1	S8 – S15 (8)
9	S8	136 572 480	2	S8 – S15 (8)	2	1	S9 – S16 (8)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
10	S9	036 152 478	2	S9 – S16 (8)	2	1	S10 – S17 (8)
11	S10	136 452 078	2	S10 – S17 (8)	2	1	S11 – S18 (8)
12	S11	130 526 478	2	S11 – S18 (8)	2	1	S12 – S19 (8)
13	S12	136 528 470	2	S12 – S19 (8)	2	1	S13 – S20 (8)
14	S13	516 032 478	3	S13 – S20 (8)	3	2	S14 – S22 (9)
15	S14	162 530 478	3	S14 – S22 (9)	3	2	S15 – S24 (10)

16	S15	136 072 548	3	S15 – S24 (10)	3	2	S16 – S26 (11)
17	S16	136 570 482	3	S16 – S26 (11)	3	2	S17 – S28 (12)
18	S17	306 152 478	3	S17 – S28 (12)	3	2	S18 – S30 (13)
19	S18	136 452 708	3	S18 – S30 (13)	3	2	S19 – S32 (14)
20	S19	103 526 478	3	S19 – S32 (14)	3	2	S20 – S34 (15)
21	S20	136 528 407	3	S20 – S34 (15)	3	2	S21 – S36 (16)
22	S21	516 432 078	4	S21 – S36 (16)	2	1	S22 – S37 (16)
23	S22	516 302 478	4	S22 – S37 (16)	4	3	S23 – S40 (18)
24	S23	162 538 470	4	S23 – S40 (18)	2	1	S24 – S41 (18)
25	S24	162 503 478	4	S24 – S41 (18)	4	3	S25 – S44 (20)
26	S25	036 172 548	4	S25 – S44 (20)	2	1	S26 – S45 (20)
27	S26	136 702 548	4	S26 – S45 (20)	4	3	S27 – S48 (22)
28	S27	130 576 482	4	S27 – S48 (22)	2	1	S28 – S49 (22)
29	S28	136 507 482	4	S28 – S49 (22)	4	3	S29 – S52 (24)
30	S29	356 102 478	4	S29 – S52 (24)	4	3	S30 – S55 (26)
31	S30	360 152 478	4	S30 – S55 (26)	2	1	S31 – S56 (26)
32	S31	136 402 758	4	S31 – S56 (26)	4	3	S32 – S59 (28)
33	S32	136 452 780	4	S32 – S59 (28)	2	1	S33 – S60 (28)
34	S33	123 506 478	4	S33 – S60 (28)	4	3	S34 – S63 (30)
35	S34	013 526 478	4	S34 – S63 (30)	2	1	S35 – S64 (30)
36	S35	136 508 427	4	S35 – S64 (30)	4	3	S36 – S67 (32)
37	S36	136 528 047	4	S36 – S67 (32)	2	1	S37 – S68 (32)
38	S37	516 432 708	5	S37 – S68 (32)	3	2	S38 – S70 (33)
39	S38	506 312 478	5	S38 – S70 (33)	3	2	S39 – S72 (34)
40	S39	516 372 408	5	S39 – S72 (34)	3	2	S40 – S74 (35)
41	S40	516 320 478	5	S40 – S74 (35)	3	2	S41 – S76 (36)
42	S41	162 538 407	5	S41 – S76 (36)	3	2	S42 – S78 (37)
43	S42	102 563 478	5	S42 – S78 (37)	3	2	S43 – S80 (38)
44	S43	162 573 408	5	S43 – S80 (38)	3	2	S44 – S82 (39)
45	S44	162 053 478	5	S44 – S82 (39)	3	2	S45 – S84 (40)
46	S45	306 172 548	5	S45 – S84 (40)	3	2	S46 – S86 (41)
47	S46	106 732 548	5	S46 – S86 (41)	3	2	S47 – S88 (42)

48	S47	136 742 508	5	S47 – S88 (42)	3	2	S48 – S90 (43)
49	S48	136 720 548	5	S48 – S90 (43)	3	2	S49 – S92 (44)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
50	S49	103 576 482	5	S49 – S92 (44)	3	2	S50 – S94 (45)
51	S50	106 537 482	5	S50 – S94 (45)	3	2	S51 – S96 (46)
52	S51	136 587 402	5	S51 – S96 (46)	3	2	S52 – S98 (47)
53	S52	136 057 482	5	S52 – S98 (47)	3	2	S53 – S100 (48)
54	S53	356 172 408	5	S53 – S100 (48)	3	2	S54 – S102 (49)
55	S54	356 012 478	5	S54 – S102 (49)	3	1	S55 – S103 (49)
56	S55	356 120 478	5	S55 – S103 (49)	3	2	S56 – S105 (50)
57	S56	362 150 478	5	S56 – S105 (50)	3	2	S57 – S107 (51)
58	S57	106 432 758	5	S57 – S107 (51)	3	2	S58 – S109 (52)
59	S58	136 042 758	5	S58 – S109 (52)	3	1	S59 – S110 (52)
60	S59	136 420 758	5	S59 – S110 (52)	3	2	S60 – S112 (53)
61	S60	136 450 782	5	S60 – S112 (53)	3	2	S61 – S114 (54)
62	S61	123 576 408	5	S61 – S114 (54)	3	2	S62 – S116 (55)
63	S62	123 056 478	5	S62 – S116 (55)	3	2	S63 – S118 (56)
64	S63	123 560 478	5	S63 – S118 (56)	3	1	S64 – S119 (56)
65	S64	513 026 478	5	S64 – S119 (56)	3	2	S65 – S121 (57)
66	S65	106 538 427	5	S65 – S121 (57)	3	2	S66 – S123 (58)
67	S66	136 058 427	5	S66 – S123 (58)	3	2	S67 – S125 (59)
68	S67	136 580 427	5	S67 – S125 (59)	3	1	S68 – S126 (59)

69	S68	136 028 547	5	S68 – S126 (59)	3	2	S69 – S128 (60)
70	S69	516 402 738	6	S69 – S128 (60)	4	3	S70 – S131 (62)
71	S70	516 432 780	6	S70 – S131 (62)	2	1	S71 – S132 (62)
72	S71	056 312 478	6	S71 – S132 (62)	2	0	S72 – S132 (61)
73	S72	560 312 478	6	S72 – S132 (61)	2	1	S73 – S133 (61)
74	S73	516 372 048	6	S73 – S133 (61)	2	1	S74 – S134 (61)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
75	S74	516 372 480	6	S74 – S134 (61)	2	1	S75 – S135 (61)
76	S75	510 326 478	6	S75 – S135 (61)	2	1	S76 – S136 (61)
77	S76	516 328 470	6	S76 – S136 (61)	2	1	S77 – S137 (61)
78	S77	162 508 437	6	S77 – S137 (61)	4	3	S78 – S140 (63)
79	S78	162 538 047	6	S78 – S140 (63)	2	1	S79 – S141 (63)
80	S79	012 563 478	6	S79 – S141 (63)	2	1	S80 – S142 (63)
81	S80	120 563 478	6	S80 – S142 (63)	2	0	S81 – S142 (62)
82	S81	162 573 048	6	S81 – S142 (62)	2	1	S82 – S143 (62)
83	S82	162 573 480	6	S82 – S143 (62)	2	1	S83 – S144 (62)
84	S83	062 153 478	6	S83 – S144 (62)	2	1	S84 – S145 (62)
85	S84	162 453 078	6	S84 – S145 (62)	2	1	S85 – S146 (62)
86	S85	376 102 548	6	S85 – S146 (62)	4	3	S86 – S149 (64)
87	S86	360 172 548	6	S86 – S149 (64)	2	1	S87 – S150 (64)

88	S87	016 732 548	6	S87 – S150 (64)	2	1	S88 – S151 (64)
89	S88	160 732 548	6	S88 – S151 (64)	2	1	S89 – S152 (64)
90	S89	136 742 058	6	S89 – S152 (64)	2	0	S90 – S152 (63)
91	S90	136 742 580	6	S90 – S152 (63)	2	1	S91 – S153 (63)
92	S91	130 726 548	6	S91 – S153 (63)	2	1	S92 – S154 (63)
93	S92	136 728 540	6	S92 – S154 (63)	2	1	S93 – S155 (63)
94	S93	173 506 482	6	S93 – S155 (63)	4	3	S94 – S158 (65)
95	S94	013 576 482	6	S94 – S158 (65)	2	1	S95 – S159 (65)
96	S95	016 537 482	6	S95 – S159 (65)	2	1	S96 – S160 (65)
97	S96	160 537 482	6	S96 – S160 (65)	2	1	S97 – S161 (65)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
98	S97	136 587 042	6	S97 – S161 (65)	2	1	S98 – S162 (65)
99	S98	136 587 420	6	S98 – S162 (65)	2	0	S99 – S162 (64)
100	S99	036 157 482	6	S99 – S162 (64)	2	1	S100 – S163 (64)
101	S100	136 457 082	6	S100 – S163 (64)	2	1	S101 – S164 (64)
102	S101	356 172 048	6	S101 – S164 (64)	2	1	S102 – S165 (64)
103	S102	356 172 480	6	S102 – S165 (64)	2	1	S103 – S166 (64)
104	S103	356 412 078	6	S103 – S166 (64)	2	1	S104 – S167 (64)
105	S104	350 126 478	6	S104 – S167 (64)	2	1	S105 – S168 (64)
106	S105	356 128 470	6	S105 – S168 (64)	2	1	S106 – S169 (64)

107	S106	362 158 470	6	S106 – S169 (64)	2	1	S107 – S170 (64)
108	S107	362 105 478	6	S107 – S170 (64)	4	3	S108 – S173 (66)
109	S108	016 432 758	6	S108 – S173 (66)	2	1	S109 – S174 (66)
110	S109	160 432 758	6	S109 – S174 (66)	2	1	S110 – S175 (66)
111	S110	036 142 758	6	S110 – S175 (66)	2	1	S111 – S176 (66)
112	S111	130 426 758	6	S111 – S176 (66)	2	1	S112 – S177 (66)
113	S112	136 428 750	6	S112 – S177 (66)	2	1	S113 – S178 (66)
114	S113	130 456 782	6	S113 – S178 (66)	2	1	S114 – S179 (66)
115	S114	136 405 782	6	S114 – S179 (66)	4	3	S115 – S182 (68)
116	S115	123 576 048	6	S115 – S182 (68)	2	1	S116 – S183 (68)
117	S116	123 576 480	6	S116 – S183 (68)	2	1	S117 – S184 (68)
118	S117	023 156 478	6	S117 – S184 (68)	2	1	S118 – S185 (68)
119	S118	123 456 078	6	S118 – S185 (68)	2	1	S119 – S186 (68)
120	S119	123 568 470	6	S119 – S186 (68)	2	1	S120 – S187 (68)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
121	S120	513 426 078	6	S120 – S187 (68)	2	1	S121 – S188 (68)
122	S121	513 206 478	6	S121 – S188 (68)	4	3	S122 – S191 (70)
123	S122	016 538 427	6	S122 – S191 (70)	2	1	S123 – S192 (70)
124	S123	160 538 427	6	S123 – S192 (70)	2	1	S124 – S193 (70)
125	S124	036 158 427	6	S124 – S193 (70)	2	1	S125 – S194 (70)

126	S125	136 458 027	6	S125 – S194 (70)	2	1	S126 – S195 (70)
127	S126	130 586 427	6	S126 – S195 (70)	2	1	S127 – S196 (70)
128	S127	036 128 547	6	S127 – S196 (70)	2	1	S128 – S197 (70)
129	S128	136 208 547	6	S128 – S197 (70)	4	3	S129 – S200 (72)
130	S129	506 412 738	7	S129 – S200 (72)	3	2	S130 – S202 (73)
131	S130	516 042 738	7	S130 – S202 (73)	3	2	S131 – S204 (74)
132	S131	516 420 738	7	S131 – S204 (74)	3	2	S132 – S206 (75)
133	S132	516 430 782	7	S132 – S206 (75)	3	2	S133 – S208 (76)
134	S133	562 310 478	7	S133 – S208 (76)	3	2	S134 – S210 (77)
135	S134	516 072 348	7	S134 – S210 (77)	3	2	S135 – S212 (78)
136	S135	516 370 482	7	S135 – S212 (78)	3	2	S136 – S214 (79)
137	S136	501 326 478	7	S136 – S214 (79)	3	2	S137 – S216 (80)
138	S137	516 328 407	7	S137 – S216 (80)	3	2	S138 – S218 (81)
139	S138	102 568 437	7	S138 – S218 (81)	3	2	S139 – S220 (82)
140	S139	162 058 437	7	S139 – S220 (82)	3	2	S140 – S222 (83)
141	S140	162 580 437	7	S140 – S222 (83)	3	2	S141 – S224 (84)
142	S141	162 038 547	7	S141 – S224 (84)	3	2	S142 – S226 (85)
143	S142	512 063 478	7	S142 – S226 (85)	3	2	S143 – S228 (86)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
144	S143	162 073 548	7	S143 – S228 (86)	3	2	S144 – S230 (87)

145	S144	162 570 483	7	S144 – S230 (87)	3	2	S145 – S232 (88)
146	S145	602 153 478	7	S145 – S232 (88)	3	2	S146 – S234 (89)
147	S146	162 453 708	7	S146 – S234 (89)	3	2	S147 – S236 (90)
148	S147	376 142 508	7	S147 – S236 (90)	3	2	S148 – S238 (91)
149	S148	376 012 548	7	S148 – S238 (91)	3	2	S149 – S240 (92)
150	S149	376 120 548	7	S149 – S240 (92)	3	2	S150 – S242 (93)
151	S150	362 170 548	7	S150 – S242 (93)	3	2	S151 – S244 (94)
152	S151	716 032 548	7	S151 – S244 (94)	3	2	S152 – S246 (95)
153	S152	162 730 548	7	S152 – S246 (95)	3	1	S153 – S247 (95)
154	S153	136 740 582	7	S153 – S247 (95)	3	2	S154 – S249 (96)
155	S154	103 726 548	7	S154 – S249 (96)	3	2	S155 – S251 (97)
156	S155	136 728 504	7	S155 – S251 (97)	3	2	S156 – S253 (98)
157	S156	173 586 402	7	S156 – S253 (98)	3	2	S157 – S255 (99)
158	S157	173 056 482	7	S157 – S255 (99)	3	2	S158 – S257 (100)
159	S158	173 560 482	7	S158 – S257 (100)	3	2	S159 – S259 (101)
160	S159	513 076 482	7	S159 – S259 (101)	3	2	S160 – S261 (102)
161	S160	516 037 482	7	S160 – S261 (102)	3	1	S161 – S262 (102)
162	S161	167 530 482	7	S161 – S262 (102)	3	2	S162 – S264 (103)
163	S162	136 087 542	7	S162 – S264 (103)	3	2	S163 – S266 (104)
164	S163	306 157 482	7	S163 – S266 (104)	3	2	S164 – S268 (105)

165	S164	136 457 802	7	S164 – S268 (105)	3	2	S165 – S270 (106)
166	S165	356 072 148	7	S165 – S270 (106)	3	2	S166 – S272 (107)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
167	S166	356 170 482	7	S166 – S272 (107)	3	1	S167 – S273 (107)
168	S167	356 412 708	7	S167 – S273 (107)	3	2	S168 – S275 (108)
169	S168	305 126 478	7	S168 – S275 (108)	3	2	S169 – S277 (109)
170	S169	356 128 407	7	S169 – S277 (109)	3	2	S170 – S279 (110)
171	S170	362 158 407	7	S170 – S279 (110)	3	2	S171 – S281 (111)
172	S171	302 165 478	7	S171 – S281 (111)	3	2	S172 – S283 (112)
173	S172	362 175 408	7	S172 – S283 (112)	3	2	S173 – S285 (113)
174	S173	362 015 478	7	S173 – S285 (113)	3	2	S174 – S287 (114)
175	S174	416 032 758	7	S174 – S287 (114)	3	2	S175 – S289 (115)
176	S175	162 430 758	7	S175 – S289 (115)	3	1	S176 – S290 (115)
177	S176	306 142 758	7	S176 – S290 (115)	3	2	S177 – S292 (116)
178	S177	103 426 758	7	S177 – S292 (116)	3	2	S178 – S294 (117)
179	S178	136 428 705	7	S178 – S294 (117)	3	2	S179 – S296 (118)
180	S179	103 456 782	7	S179 – S296 (118)	3	2	S180 – S298 (119)
181	S180	106 435 782	7	S180 – S298 (119)	3	2	S181 – S300 (120)
182	S181	136 485 702	7	S181 – S300 (120)	3	2	S182 – S302 (121)
183	S182	136 045 782	7	S182 – S302 (121)	3	2	S183 – S304 (122)

184	S183	123 076 548	7	S183 – S304 (122)	3	1	S184 – S305 (122)
185	S184	123 570 486	7	S184 – S305 (122)	3	2	S185 – S307 (123)
186	S185	203 156 478	7	S185 – S307 (123)	3	2	S186 – S309 (124)
187	S186	123 456 708	7	S186 – S309 (124)	3	1	S187 – S310 (124)
188	S187	123 568 407	7	S187 – S310 (124)	3	2	S188 – S312 (125)
189	S188	513 426 708	7	S188 – S312 (125)	3	2	S189 – S314 (126)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
190	S189	503 216 478	7	S189 – S314 (126)	3	2	S190 – S316 (127)
191	S190	513 276 408	7	S190 – S316 (127)	3	2	S191 – S318 (128)
192	S191	513 260 478	7	S191 – S318 (128)	3	2	S192 – S320 (129)
193	S192	516 038 427	7	S192 – S320 (129)	3	1	S193 – S321 (129)
194	S193	168 530 427	7	S193 – S321 (129)	3	2	S194 – S323 (130)
195	S194	306 158 427	7	S194 – S323 (130)	3	1	S195 – S324 (130)
196	S195	136 458 207	7	S195 – S324 (130)	3	2	S196 – S326 (131)
197	S196	103 586 427	7	S196 – S326 (131)	3	2	S197 – S328 (132)
198	S197	306 128 547	7	S197 – S328 (132)	3	2	S198 – S330 (133)
199	S198	106 238 547	7	S198 – S330 (133)	3	2	S199 – S332 (134)
200	S199	136 248 507	7	S199 – S332 (134)	3	2	S200 – S334 (135)
201	S200	136 280 547	7	S200 – S334 (135)	3	2	S201 – S336 (136)
202	S201	056 412 738	8	S201 – S336 (136)	2	1	S202 – S337 (136)

203	S202	560 412 738	8	S202 – S337 (136)	2	1	S203 – S338 (136)
204	S203	016 542 738	8	S203 – S338 (136)	2	1	S204 – S339 (136)
205	S204	516 742 038	8	S204 – S339 (136)	2	1	S205 – S340 (136)
206	S205	510 426 738	8	S205 – S340 (136)	2	1	S206 – S341 (136)
207	S206	516 428 730	8	S206 – S341 (136)	2	1	S207 – S342 (136)
208	S207	510 436 782	8	S207 – S342 (136)	2	1	S208 – S343 (136)
209	S208	516 403 782	8	S208 – S343 (136)	4	3	S209 – S346 (138)
210	S209	562 318 470	8	S209 – S346 (138)	2	1	S210 – S347 (138)
211	S210	562 301 478	8	S210 – S347 (138)	4	3	S211 – S350 (140)
212	S211	016 572 348	8	S211 – S350 (140)	2	1	S212 – S351 (140)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
213	S212	516 702 348	8	S212 – S351 (140)	4	2	S213 – S353 (141)
214	S213	510 376 482	8	S213 – S353 (141)	2	1	S214 – S354 (141)
215	S214	516 307 482	8	S214 – S354 (141)	4	2	S215 – S356 (142)
216	S215	521 306 478	8	S215 – S356 (142)	4	3	S216 – S359 (144)
217	S216	051 326 478	8	S216 – S359 (144)	2	1	S217 – S360 (144)
218	S217	516 308 427	8	S217 – S360 (144)	4	2	S218 – S362 (145)
219	S218	516 328 047	8	S218 – S362 (145)	2	1	S219 – S363 (145)
220	S219	012 568 437	8	S219 – S363 (145)	2	1	S220 – S364 (145)
221	S220	120 568 437	8	S220 – S364 (145)	2	1	S221 – S365 (145)

222	S221	062 158 437	8	S221 – S365 (145)	2	1	S222 – S366 (145)
223	S222	162 458 037	8	S222 – S366 (145)	2	1	S223 – S367 (145)
224	S223	160 582 437	8	S223 – S367 (145)	2	1	S224 – S368 (145)
225	S224	162 587 430	8	S224 – S368 (145)	2	1	S225 – S369 (145)
226	S225	062 138 547	8	S225 – S369 (145)	2	1	S226 – S370 (145)
227	S226	162 308 547	8	S226 – S370 (145)	4	3	S227 – S373 (147)
228	S227	512 463 078	8	S227 – S373 (147)	2	1	S228 – S374 (147)
229	S228	512 603 478	8	S228 – S374 (147)	4	3	S229 – S377 (149)
230	S229	062 173 548	8	S229 – S377 (149)	2	1	S230 – S378 (149)
231	S230	162 703 548	8	S230 – S378 (149)	4	2	S231 – S380 (150)
232	S231	160 572 483	8	S231 – S380 (150)	2	1	S232 – S381 (150)
233	S232	162 507 483	8	S232 – S381 (150)	4	2	S233 – S383 (151)
234	S233	652 103 478	8	S233 – S383 (151)	4	3	S234 – S386 (153)
235	S234	620 153 478	8	S234 – S386 (153)	2	1	S235 – S387 (153)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
236	S235	162 403 758	8	S235 – S387 (153)	4	2	S236 – S389 (154)
237	S236	162 453 780	8	S236 – S389 (154)	2	1	S237 – S390 (154)
238	S237	376 142 058	8	S237 – S390 (154)	2	1	S238 – S391 (154)
239	S238	376 142 580	8	S238 – S391 (154)	2	1	S239 – S392 (154)
240	S239	076 312 548	8	S239 – S392 (154)	2	1	S240 – S393 (154)

241	S240	376 512 048	8	S240 – S393 (154)	2	1	S241 – S394 (154)
242	S241	370 126 548	8	S241 – S394 (154)	2	1	S242 – S395 (154)
243	S242	376 128 540	8	S242 – S395 (154)	2	1	S243 – S396 (154)
244	S243	362 178 540	8	S243 – S396 (154)	2	1	S244 – S397 (154)
245	S244	362 107 548	8	S244 – S397 (154)	4	3	S245 – S400 (156)
246	S245	716 532 048	8	S245 – S400 (156)	2	1	S246 – S401 (156)
247	S246	716 302 548	8	S246 – S401 (156)	4	2	S247 – S403 (157)
248	S247	162 738 540	8	S247 – S403 (157)	2	1	S248 – S404 (157)
249	S248	130 746 582	8	S248 – S404 (157)	2	1	S249 – S405 (157)
250	S249	136 704 582	8	S249 – S405 (157)	4	3	S250 – S408 (159)
251	S250	123 706 548	8	S250 – S408 (159)	4	2	S251 – S410 (160)
252	S251	013 726 548	8	S251 – S410 (160)	2	1	S252 – S411 (160)
253	S252	136 708 524	8	S252 – S411 (160)	4	3	S253 – S414 (162)
254	S253	136 728 054	8	S253 – S414 (162)	2	1	S254 – S415 (162)
255	S254	173 586 042	8	S254 – S415 (162)	2	1	S255 – S416 (162)
256	S255	173 586 420	8	S255 – S416 (162)	2	1	S256 – S417 (162)
257	S256	073 156 482	8	S256 – S417 (162)	2	1	S257 – S418 (162)
258	S257	173 456 082	8	S257 – S418 (162)	2	1	S258 – S419 (162)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
259	S258	170 563 482	8	S258 – S419 (162)	2	1	S259 – S420 (162)

260	S259	173 562 480	8	S259 – S420 (162)	2	1	S260 – S421 (162)
261	S260	513 476 082	8	S260 – S421 (162)	2	1	S261 – S422 (162)
262	S261	513 706 482	8	S261 – S422 (162)	4	3	S262 – S425 (164)
263	S262	516 437 082	8	S262 – S425 (164)	2	1	S263 – S426 (164)
264	S263	167 532 480	8	S263 – S426 (164)	2	1	S264 – S427 (164)
265	S264	167 503 482	8	S264 – S427 (164)	4	2	S265 – S429 (165)
266	S265	036 187 542	8	S265 – S429 (165)	2	1	S266 – S430 (165)
267	S266	136 807 542	8	S266 – S430 (165)	4	3	S267 – S433 (167)
268	S267	356 107 482	8	S267 – S433 (167)	4	2	S268 – S435 (168)
269	S268	360 157 482	8	S268 – S435 (168)	2	1	S269 – S436 (168)
270	S269	136 407 852	8	S269 – S436 (168)	4	3	S270 – S439 (170)
271	S270	136 457 820	8	S270 – S439 (170)	2	1	S271 – S440 (170)
272	S271	056 372 148	8	S271 – S440 (170)	2	1	S272 – S441 (170)
273	S272	356 702 148	8	S272 – S441 (170)	4	3	S273 – S444 (172)
274	S273	350 176 482	8	S273 – S444 (172)	2	1	S274 – S445 (172)
275	S274	356 402 718	8	S274 – S445 (172)	4	3	S275 – S448 (174)
276	S275	356 412 780	8	S275 – S448 (174)	2	1	S276 – S449 (174)
277	S276	325 106 478	8	S276 – S449 (174)	4	3	S277 – S452 (176)
278	S277	035 126 478	8	S277 – S452 (176)	2	1	S278 – S453 (176)
279	S278	356 108 427	8	S278 – S453 (176)	4	2	S279 – S455 (177)

280	S279	356 128 047	8	S279 – S455 (177)	2	1	S280 – S456 (177)
281	S280	362 108 457	8	S280 – S456 (177)	4	3	S281 – S459 (179)

Iter.	State	Current State	Depth	Queue Before (size)	Gen.	New	Queue After (size)
282	S281	362 158 047	8	S281 – S459 (179)	2	1	S282 – S460 (179)
283	S282	032 165 478	8	S282 – S460 (179)	2	1	S283 – S461 (179)
284	S283	320 165 478	8	S283 – S461 (179)	2	0	S284 – S461 (178)
285	S284	362 175 048	8	S284 – S461 (178)	2	1	S285 – S462 (178)
286	S285	362 175 480	8	S285 – S462 (178)	2	1	S286 – S463 (178)
287	S286	062 315 478	8	S286 – S463 (178)	2	1	S287 – S464 (178)
288	S287	362 415 078	8	S287 – S464 (178)	2	1	S288 – S465 (178)
289	S288	416 732 058	8	S288 – S465 (178)	2	1	S289 – S466 (178)
290	S289	416 302 758	8	S289 – S466 (178)	4	3	S290 – S469 (180)
291	S290	162 438 750	8	S290 – S469 (180)	2	1	S291 – S470 (180)
292	S291	346 102 758	8	S291 – S470 (180)	4	3	S292 – S473 (182)
293	S292	360 142 758	8	S292 – S473 (182)	2	1	S293 – S474 (182)
294	S293	123 406 758	8	S293 – S474 (182)	4	2	S294 – S476 (183)
295	S294	013 426 758	8	S294 – S476 (183)	2	1	S295 – S477 (183)
296	S295	136 408 725	8	S295 – S477 (183)	4	3	S296 – S480 (185)
297	S296	136 428 075	8	S296 – S480 (185)	2	1	S297 – S481 (185)
298	S297	153 406 782	8	S297 – S481 (185)	4	3	S298 – S484 (187)

299	S298	013 456 782	8	S298 – S484 (187)	2	1	S299 – S485 (187)
300	S299	016 435 782	8	S299 – S485 (187)	2	1	S300 – S486 (187)
301	S300	160 435 782	8	S300 – S486 (187)	2	1	S301 – S487 (187)
302	S301	136 485 072	8	S301 – S487 (187)	2	1	S302 – S488 (187)
303	S302	136 485 720	8	S302 – S488 (187)	2	0	S303 – S488 (186)
304	S303	036 145 782	8	S303 – S488 (186)	2	1	S304 – S489 (186)
Iter.	**State**	**Current State**	**Depth**	**Queue Before (size)**	**Gen.**	**New**	**Queue After (size)**
305	S304	136 745 082	8	S304 – S489 (186)	2	1	S305 – S490 (186)
306	S305	023 176 548	8	S305 – S490 (186)	2	1	S306 – S491 (186)
307	S306	120 573 486	8	S306 – S491 (186)	2	1	S307 – S492 (186)
308	S307	123 507 486	8	S307 – S492 (186)	4	3	S308 – S495 (188)
309	S308	253 106 478	8	S308 – S495 (188)	4	3	S309 – S498 (190)
310	S309	230 156 478	8	S309 – S498 (190)	2	1	S310 – S499 (190)
311	S310	123 456 780	8	S310 – S499 (190)	0	0	GOAL – stop

At iteration 311 the goal state S310 = 123 456 780 is at the front of the queue. It is removed, found to be the goal, and the search stops.

3.6 Final Answers

(a) Which node/state is expanded in each iteration?

BFS expands the states in the order in which they were discovered: iteration k expands state S(k−1). Thus iteration 1 expands S0 (the initial state 136 502 478), iteration 2 expands S1, iteration 3 expands S2, iteration 4 expands S3 and iteration 5 expands S4, and so on up to iteration 311, which removes S310 (the goal) from the queue. The states expanded in iterations 1–5 are shown in Table A, and the complete sequence is given in Table C.

(b) How many states are generated in total?

Measure (operator order: Up, Down, Left, Right)	Count

States expanded (removed from queue and expanded; the goal is removed but not expanded)	310
States removed from the queue (including the goal)	311
Successors generated by legal moves (including duplicates)	838
Distinct states discovered (S0 – S499, including the initial state)	500
Discovered states still waiting in the queue when the goal is dequeued	189
Duplicate successors discarded by the visited set	339

Answer: BFS **generates 838 successor states** in total (499 of them are new and 339 are duplicates rejected by the visited set). Including the initial state, **500 distinct states are discovered**, and **311 states are removed from the queue** (310 expanded before the goal is dequeued). These counts depend on the operator order Up, Down, Left, Right; the solution length of 8 moves does not.

(c) Complete solution path from the initial state to the goal state

The solution has **8 moves**: Right → Up → Left → Down → Left → Down → Right → Right (moves of the blank tile).

State numbers along the path: S0 → S4 → S11 → S19 → S33 → S62 → S118 → S186 → S310.

State	State	State
Step 0 – Initial 1 3 6 5 _ 2 4 7 8	Step 1 – Right 1 3 6 5 2 _ 4 7 8	Step 2 – Up 1 3 _ 5 2 6 4 7 8
Step 3 – Left 1 _ 3 5 2 6 4 7 8	Step 4 – Down 1 2 3 5 _ 6 4 7 8	Step 5 – Left 1 2 3 _ 5 6 4 7 8
Step 6 – Down 1 2 3 4 5 6 _ 7 8	Step 7 – Right 1 2 3 4 5 6 7 _ 8	Step 8 – Right 1 2 3 4 5 6 7 8 _

(Blank tile shown as "_". Each arrow in the sequence is a move of the blank.)

(d) Why does BFS always find the shortest solution in an unweighted 8-puzzle?

Every move in the 8-puzzle has the same cost (1), so the cost of a solution equals the number of moves. BFS expands all states at depth d before any state at depth d + 1, because its queue is FIFO. Hence the first time the goal state is reached, it cannot have been reachable at a smaller depth – otherwise it would already have been discovered earlier. The first solution found is therefore the one with the fewest moves. The visited set does not affect this, since it

only removes states that were already reached by an equal or shorter path. Here the goal is at depth 8, so no solution with fewer than 8 moves exists.

3.7 Explanation and Result

Starting from the initial state (blank at the centre), BFS expanded states level by level using the Up, Down, Left, Right operator order and a visited set to reject duplicates. The goal state 123 456 780 was found at depth 8 after 311 iterations. Verification by following the path above, each move is a legal blank swap and the last grid equals the goal state exactly.

Number of states per depth (discovered): depth 0: 1, depth 1: 4, depth 2: 8, depth 3: 8, depth 4: 16, depth 5: 32, depth 6: 60, depth 7: 72, depth 8: 136, depth 9: 163.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.